

AI導入支援エージェント × AIWBS作成エージェント 実装コード評価レポート

評価日：2026年7月27日 | 評価者：Claude | 対象：AI-Agent-Dev-WBS-OS リポジトリ（実ソースコードの直接レビュー＋独自実行検証）

本レポートは、開発ログ（チャット記録）の自己申告ではなく、実際のソースコードを読み、一部はこちらの環境で実際に実行・再現して検証した結果に基づく。評価軸は依頼された次の3点。

- ① 公開・販売するに値する質か
- ② 安全設計（HITL・監査ログ）がWBS通りに実装に反映されているか
- ③ 他の目的にも汎用的に使える作りになっているか

① 公開・販売するに値する質か

結論：Yes。実務水準のコード品質であることを、独自の実行検証で確認した。

根拠

アルゴリズム	F60（MECEチェック）はコサイン類似度、F70（階層生成）・F80（トレーサビリティ）はUnion-Findを外部ライブラリなしで自前実装。プロンプト頼みではなく、適切なアルゴリズム設計が行われている。
独自の実行検証	F20～F90（外部API不要）をこちらでimportし、過去に保存されたF10出力を実際にF20→F90まで通してパイプライン実行。エラーなく完走し、各段で期待通りのキー構造（trace_id / hitl / hitl_required 等）を確認。MECEチェックが実際に「非準拠」警告を出すことも確認済み。
テスト網羅性	バックアップレポートによれば500関数・590ケースが全通過。F20～F90はこちらでも独立してimport・実行を確認できた（anthropicライブラリが使えない本サンドボックスの制約上、pytest自体は実行できなかったため、主要シナリオを手動再現して検証）。
エラー処理・堅牢性	リトライ処理、例外の__cause__チェーン、型ヒント、Args/Returns/RaisesまでのDocstringなど、実務水準のPython作法に沿っている。
セキュリティ	.envは空欄のまま管理され、実際のAPIキー漏洩は確認されなかった。バックアップレポート内の「sk-ant-...」もプレースホルダーであり実キーではない。
発見・修正した不具合	f10_module.pyの_call_api_with_retryが、API呼び出しエラーのみリトライ対象とし、不正なJSON応答時はリトライされず1回で失敗する実装になっていた。設計意図（3回リトライ）とズレていたため、このセッション内で修正し、既存テスト契約を壊さないことをシナリオ再現で確認した。

率直なコメント：チャット越しの自己申告ではなく、実際に動くコードとして確認できた点が多い。見つかった不具合も軽微かつ既に修正済みであり、全体として公開・販売に見合う水準にあると判断する。

② 安全設計（HITL・監査ログ）がWBS通りに実装に反映されているか

結論：Yes。WBSの理念が、実際に動くコードとして実装されていることを確認した。

根拠

HITL判定の実装	f10_module.py内に曖昧語検出（「など」「いろいろ」等）・粒度不足判定（文字数閾値）が実際のコードとして存在し、execute()実行時に自動でHITL移譲を返す。WBS文書内の理念記述だけでなく、動くロジックとして確認できた。
HITLTracker	承認／却下／再処理を記録するクラスが実装されており、承認率の計算、誤承認警告（承認率90%超で発火）、保留超過検知（delay_seconds指定で保留中案件を抽出）を提供。CLAUDE.mdに記載の「誤承認検知閾値：承認率＞90%」「HITL承認遅延30秒超過で通知」という仕様値と実装が一致している。
MonitoringHandler （監査ログ）	Pythonの標準loggingフレームワークに実際にフックする本物のHandlerクラス。F10～F90全モジュールのロガーに接続し、ERROR即時／WARNING5件連続／RETRY3件連続／HITLの4分類でアラートを発火し、実ファイル（docs/phase4/logs/summary.log）に追記する。CLAUDE.mdのアラートルール定義と実装が一致。
実際の監査ログの実在確認	docs/phase4/logs/summary.logを直接確認したところ、WP8100～WP8330の各テスト区分ごとにpassed/skip/failed件数が記録された、実行環境情報付きの構造化ログが実在した。

率直なコメント：WBS文書に書かれた安全思想（最小権限・HITL必須化・監査ログ・誤承認検知）が、「言葉だけ」ではなく実際に動作するPythonコードとして裏付けられている点は、この実装の一番の強みだと考える。

③ 他の目的にも汎用的に使える作りになっているか

結論：基本設計としては汎用的。ただし検証済みの適用範囲はまだ限定的。

根拠

プロンプトの汎用性	F10のシステムプロンプト（f10_system.txt）は「与えられた目的文」を分析する構造で書かれており、特定業務（為替予測など）に限定した記述は一切ない。L1/L2/L3の階層化ルールも業種非依存。
テンプレートの汎用性	F50の優先度別テンプレート（TMP_HIGH/MEDIUM/LOW）は「優先度」という抽象軸に基づく汎用フォーマットであり、特定ドメインの語彙にロックインされていない。
既に確認できている適用範囲	為替予測（プロトコル型のロールプレイ運用）に加え、テストコード内では「新規顧客獲得を強化し、売上を前年比120%に成長させる」という別ドメイン（営業目標）の例でF10～F90が正しく動作することも確認できた。
運用コスト面の汎用性	外部APIが必要なのはF10のみで、F20以降は無料・高速・決定的に動く。ドメインを変えて繰り返し試す際のコスト障壁が低い設計になっている。

留保点（正直な限界）

「汎用的に使える設計」であることと、「あらゆる目的で実際に検証済み」であることは別。現時点で動作確認できているのは為替予測・営業目標という2種類の例にとどまる。書籍やGitHubで「どんな目的にも使える」と強く打ち出す場合は、性質の異なるテーマ（例：自治体の業務改善、個人のキャリア目標、非営利活動の企画など）でも1～2件試し、同様に破綻なく動くことを示しておく、汎用性の主張により説得力が増す。

総合結論

3つの評価軸のうち、①公開・販売するに値する質か、②HITL・監査ログの実装反映、の2点は実コードと独自の実行検証によって明確にYesと判断できる。③の汎用性は設計思想としては汎用的だが、実証データがまだ2ケースにとどまるため、「条件付きYes」として扱うのが正確。

総じて、開発ログの自己申告だけに頼らず、実際に動くコードとして確認できたことは大きい。見つかった不具合（f10_module.pyのリトライ処理）は軽微であり、このセッション内で修正済み。AI導入支援エージェント側も、以前の指摘を受けて .claude/agents/ai-donyu-shien.md として正式なエージェント定義ファイルに整理済みであり、AIWBS作成エージェントと並ぶ「見せられる実体」になっている。

※本評価は、F10～F90・monitoring配下の主要モジュールを中心にサンプリングレビューしたものであり、improvement/knowledge/deployment/phase9/phase10配下を含む全40以上のソースファイルを1行単位で網羅したものではない。また本サンドボックス環境はネットワーク制約によりpytest／anthropicライブラリを導入できず、実際のpytest実行結果はお手元での「pytest tests/ -v」実行による最終確認を推奨する。

追記（2026年7月28日）：Phase 7～10 再レビューでの追加発見

本レポート作成後、リスクの高い領域として Phase 8～10（展開層・完成層・運用監視層）、続いて Phase 7（学習層）を追加でサンプルレビューした。いずれも「公開・販売するに値する質か」という①の結論（Yes）を覆すものではないが、根拠として開示しておくべき共通パターンが見つかったため、ここに追記する。

共通パターン：判断ロジック・閾値評価・HITL承認ゲート・監査ログの記録はいずれも実際に動作する本物の実装である。一方で、各モジュール名が示す「本質的な部分」（実インフラへの操作、実データに基づく学習）はまだプレースホルダーであり、今後の実装が必要な拡張ポイントとして残っている。

発見1：Phase 8～10の「実行アクション」はシミュレーション

デプロイ（f9510/f9520）、負荷テスト（f9530）、ロールバック実行（f10140、f9520）、自律運用トライアル（f9620）は、いずれもコード内のコメントで「決定論的シミュレーション」と明記されており、実際の本番環境・インフラに対する操作は行われず。本番運用でこれらのモジュールを使う場合は、シミュレーション部分を実際のデプロイ／ロールバック処理に置き換える実装が別途必要。

発見2：Phase 7の「自己最適化指数」はルールベースの分類スコア

学習パターンの「再現性」ラベルは、データの出所ごとに固定ルールで割り当てられており（実際に複数回試行して測定した値ではない）、それを固定の点数表（high=1.0／medium=0.7／low=0.4等）とカテゴリ別の固定重みで加重平均したものが「最適化指数（opt_avg=0.9119等）」である。小数点4桁の数値は統計的に測定された精度を意味するものではなく、あらかじめ設計された採点ルールの機械的な適用結果である。

①の結論への影響

「公開・販売するに値する質か」の結論（Yes）は変わらない。ただし、これらの限界を開示せずに「自律運用」「自己最適化」という言葉だけを前面に出すと、コードを読んだ読者・購入者との間で信頼を損なうリスクがある。GitHub公開用README（Known Limitations／既知の制限事項）には、この追記内容を反映済み。書籍・販売時の説明でも同様の開示を行うことを推奨する。